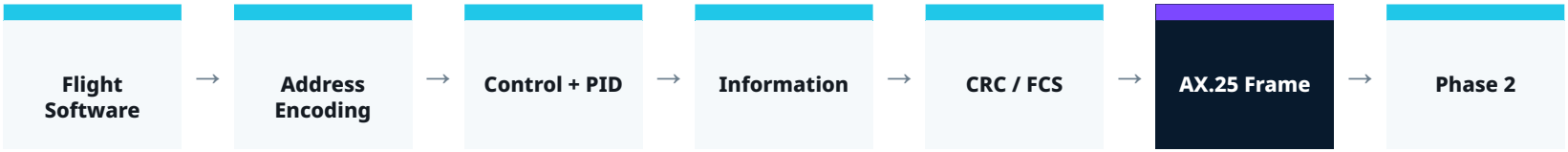
Phase 1 - AX.25 Frame Core

จาก Mission Data สู่ AX.25 UI Frame ระดับ Byte - เห็นเส้นทางข้อมูลก่อนลงรายละเอียด

1 Block Diagram - ข้อมูลไหลอย่างไร?

รับข้อมูลจาก Flight Software → สร้าง AX.25 frame → ส่งต่อให้ Phase 2 ทำ Bit / HDLC



INPUT - รับมาจากไหน?

Flight Software ส่งข้อมูลที่ต้องการสื่อสาร เช่น housekeeping telemetry. ตัวอย่าง prototype: `SEQ=1,BAT=7.60,TEMP=31.4,MODE=NOMINAL`

OUTPUT - ส่งต่อให้ใคร?

Phase 1 สร้าง **logical AX.25 frame** ที่ตรวจ CRC ได้แล้ว จากนั้น Phase 2 จะนำ frame body ไปแปลงเป็น bitstream, stuffing, HDLC และ NRZI.

ขอบเขตของ Phase 1: ทำงานระดับ **Byte / Frame**. ยังไม่ทำ Bit Stuffing, HDLC flag stream, NRZI หรือ RF modulation.

FLAG	DESTINATION	SOURCE	CONTROL	PID	INFORMATION	FCS	FLAG
7E	GROUND-0	ORBITL-1	03	F0	Telemetry	E2 E1	7E

GROUND-0 / ORBITL-1 เป็น test identifiers สำหรับ prototype เท่านั้น ยังไม่ใช่ final mission callsign.

2 แต่ละ Block ทำอะไร และใช้ตอนไหน?

อ่านจากซ้ายไปขวาตาม data flow ของหน้าแรก

1) Address Encoding

ทำอะไร: แปลง callsign + SSID เป็น AX.25 address field 7 bytes ต่อสถานี.

ใช้ตอนไหน: ทุกครั้งที่สร้าง frame เพื่อระบุว่า “ส่งจากใคร → ไปหาใคร”.

ตัวอย่าง:

GROUND-0 → `8E A4 9E AA 9C 88 60`

ORBITL-1 → `9E A4 84 92 A8 98 63`

2) Control Field

ทำอะไร: บอกชนิด/พฤติกรรมของ AX.25 frame.

ใช้ตอนไหน: เมื่อ receiver ต้องตีความว่า frame นี้เป็นรูปแบบใด.

Prototype: ใช้ `0x03` = UI (Unnumbered Information) frame เพื่อส่งข้อมูลโดยไม่ทำ connection-oriented sequencing ในชั้นนี้.

3) PID - Protocol ID

ทำอะไร: ระบุ protocol ของ payload ที่อยู่ใน Information Field.

ใช้ตอนไหน: receiver อ่านต่อจาก Control field เพื่อรู้ว่าจะตีความ payload ต่ออย่างไร.

Prototype: `0xF0` = no layer-3 protocol.

4) Information Field

ทำอะไร: เป็นพื้นที่เก็บข้อมูลจริงที่เราต้องการส่ง.

ใช้ตอนไหน: หลัง header fields ถูกกำหนดแล้ว ก่อนคำนวณ CRC.

ตอนนี้: ใช้ ASCII telemetry เพื่อทดสอบ software core. Phase 3 จะเปลี่ยนเป็น ORBITL-1 binary mission packet.

5) CRC-16/X-25 → FCS

ทำอะไร: ตรวจจับความผิดพลาดของข้อมูลใน frame body.

ใช้ตอนไหน: Sender คำนวณก่อนส่ง; receiver คำนวณซ้ำเมื่อรับ frame.

Known result: CRC numeric = `0xE1E2`; วางใน frame แบบ low-byte-first เป็น `E2 E1`.

6) Frame Assembly / Decoder

ทำอะไร: รวม Address + Control + PID + Information + FCS เป็น frame และ decoder แยกกลับ.

ใช้ตอนไหน: ก่อนส่งต่อ Phase 2 และเมื่อ Ground side ต้องตรวจ/อ่านข้อมูลกลับ.

ผล: logical frame ตัวอย่างยาว **57 bytes** รวม opening/closing flag.

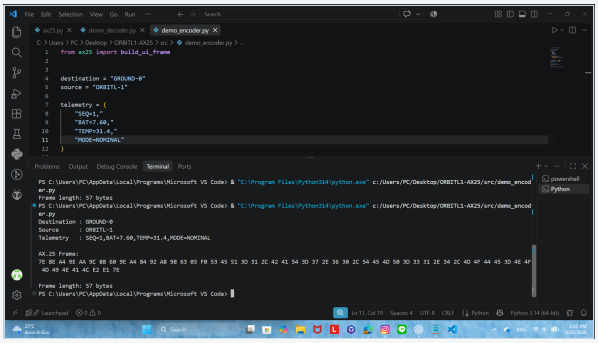
เห็นภาพง่าย ๆ: Address = หน้าซอง, Control/PID = ป้ายบอกประเภท, Information = ของในซอง, FCS = ซีลตรวจว่าซองถูกแก้หรือเสียหายระหว่างทางหรือไม่.

CRC ไม่ได้แก้ error: มันทำหน้าที่ “ตรวจจับ” ว่าข้อมูลที่รับมาไม่ตรงกับที่ sender คำนวณไว้ แล้ว decoder จึง reject frame.

3 Code → Frame: โค้ดส่วนไหนทำ Block ไหน?

เลือกเฉพาะ logic สำคัญ แล้ววางคู่กับหลักฐานจาก terminal

```
# CRC-16/X-25 : สร้าง FCS ของ frame body
def crc16_x25(data: bytes) -> int:
    crc = 0xFFFF
for byte in data:
        crc ^= byte
        for _ in range(8):
            if crc & 1:
                crc = (crc >> 1) ^ 0x8408
else:
                crc >>= 1
return crc ^ 0xFFFF
# ประกอบ logical AX.25 UI frame
def build_ui_frame(destination, source, info):
    body = (
        encode_address(destination, last=False)
        + encode_address(source, last=True)
        + bytes([0x03, 0xF0])
        + info
    )
    fcs = crc16_x25(body)
    return bytes([0x7E]) + body         + fcs.to_bytes(2,
"little")         + bytes([0x7E])
```



Encoder evidence: telemetry prototype ถูกนำมาสร้าง AX.25 frame ได้จริง และแสดง byte sequence ของ frame ใน terminal.

Mapping กับ Block Diagram:

`encode_address()` → Address Encoding

`[0x03, 0xF0]` → Control + PID

`info` → Information Field

`crc16_x25()` → CRC / FCS

`build_ui_frame()` → Frame Assembly

1 `crc = 0xFFFF`

ค่าเริ่มต้นของ CRC-16/X-25. Sender และ receiver ต้องเริ่มจากค่าเดียวกันเพื่อให้ผลตรวจตรงกัน.

2 `0x8408`

เป็น reflected polynomial ของ CRC-16/X-25 ที่ใช้กับการประมวลผลแบบ LSB-first ใน loop นี้.

3 `last=False / last=True`

Destination ยังไม่จบ address field; Source เป็น address ตัวสุดท้าย จึงตั้ง extension bit ที่ source.

4 `[0x03, 0xF0]`

`0x03` คือ UI frame และ `0xF0` คือ no layer-3 protocol ตาม prototype baseline.

5 `crc16_x25(body)`

คำนวณ CRC เฉพาะ frame body - ไม่รวม opening/closing `0x7E` flags.

6 `to_bytes(2, "little")`

CRC numeric `0xE1E2` ถูกวางใน frame เป็น E2 E1; logical frame รวม flags ยาว 57 bytes.

4 Decode + Error Test: เราพิสูจน์ Phase 1 อย่างไร?

ไม่ใช่แค่สร้าง frame ได้ แต่ต้องอ่านกลับได้และตรวจ frame เสียได้

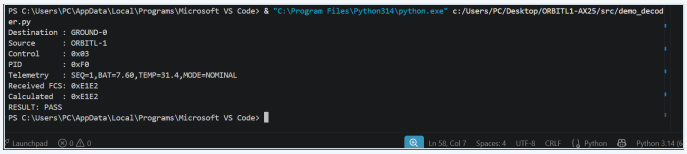
```python
# Receiver-side validation (แนวคิดหลัก)
def decode_ui_frame(frame: bytes):
    if frame[0] != 0x7E or frame[-1] != 0x7E:
        raise ValueError("Invalid AX.25 flags")

    body = frame[1:-3]
    received_fcs = int.from_bytes(frame[-3:-1], "little")
    calculated_fcs = crc16_x25(body)

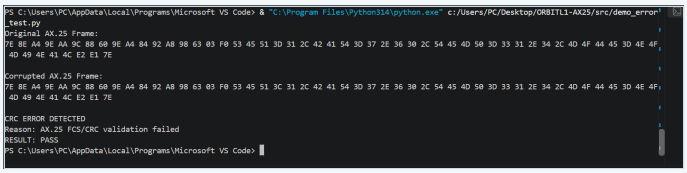
    if received_fcs != calculated_fcs:
        raise ValueError("AX.25 FCS/CRC validation failed")

    return parse_fields(body)

# Fault injection test: เปลี่ยนข้อมูลเพียง 1 bit
corrupted = bytearray(frame)
corrupted[20] ^= 0x01
decode_ui_frame(bytes(corrupted))
```



Decoder PASS: อ่าน Destination / Source / Control / PID / Telemetry กลับได้ และ Received FCS = Calculated FCS.



CRC Error Test: เมื่อทำ fault injection 1 bit ระบบตรวจพบ CRC mismatch และรายงาน PASS เพราะ reject frame ได้ตามคาด.

สิ่งที่ test นี้พิสูจน์: ถ้า frame ถูกเปลี่ยนแม้เพียง 1 bit ค่า CRC ที่ receiver คำนวณจะไม่ตรงกับ FCS ที่มากับ frame และ decoder จะ reject แทนการส่งข้อมูลผิดขึ้น Ground Software.

1 ตรวจ `0x7E` ก่อน
ถ้า opening/closing flag ไม่ถูกต้อง decoder จะหยุดทันที ไม่ตีความ frame ต่อ.

2 `body = frame[1:-3]`
ตัด opening flag ออก และเว้น 2-byte FCS + closing flag เพื่อให้เหลือเฉพาะ body ที่ใช้คำนวณ CRC.

3 `int.from_bytes(..., "little")`
อ่าน FCS จาก frame ให้กลับมาเป็นค่าตัวเลข CRC ที่นำไปเปรียบเทียบได้.

4 คำนวณซ้ำแล้วเปรียบเทียบ
ถ้า `received_fcs != calculated_fcs` ให้ reject frame - CRC ทำหน้าที่ตรวจจับ ไม่ได้ซ่อมข้อมูล.

5 `corrupted[20] ^= 0x01`
สลับเพียง 1 bit เพื่อจำลอง transmission error แล้วคาดหวังให้ CRC validation ล้มเหลว.

6 ความหมายของ PASS
Decoder อ่าน frame ปกติได้ และปฏิเสธ frame ที่ถูกแก้ 1 bit ได้ตามที่ออกแบบ.

✓ Phase 1 Exit Evidence

สิ่งที่ผ่านก่อนส่งต่อให้ Phase 2

- ✓ สร้าง AX.25 UI frame ตามโครงที่กำหนดได้
- ✓ Known FCS `0xE1E2` ตรงตาม test vector
- ✓ 1-bit corruption ถูก CRC ตรวจพบและ reject
- ✓ Address encode/decode round trip ได้
- ✓ Decoder อ่านข้อมูลเดิมกลับได้
- ✓ Phase 1 automated tests: 13 tests / OK

ส่งต่อ Phase 2: ถอด logical flags ออก เหลือ **frame body 55 bytes** เพื่อทำ LSB-first serialization → bit stuffing → HDLC flags → NRZI.